

Vorlesung Cybersicherheit, Sommersemester 2025

Sicherheitsprotokolle, Denial-of-Service, Input Validation im Web

Prof. Dr.-Ing. Lucas Vincenzo Davi

Fakultät für Informatik

Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>

Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi

Rückblick und Themen heute

- Rückblick
 - Diffie-Hellman Key Exchange
 - Needham-Schroeder Protokoll
 - Kerberos
- Themen heute
 - LVB
 - SSL/TLS
 - Denial-of-Service
 - Input Validation



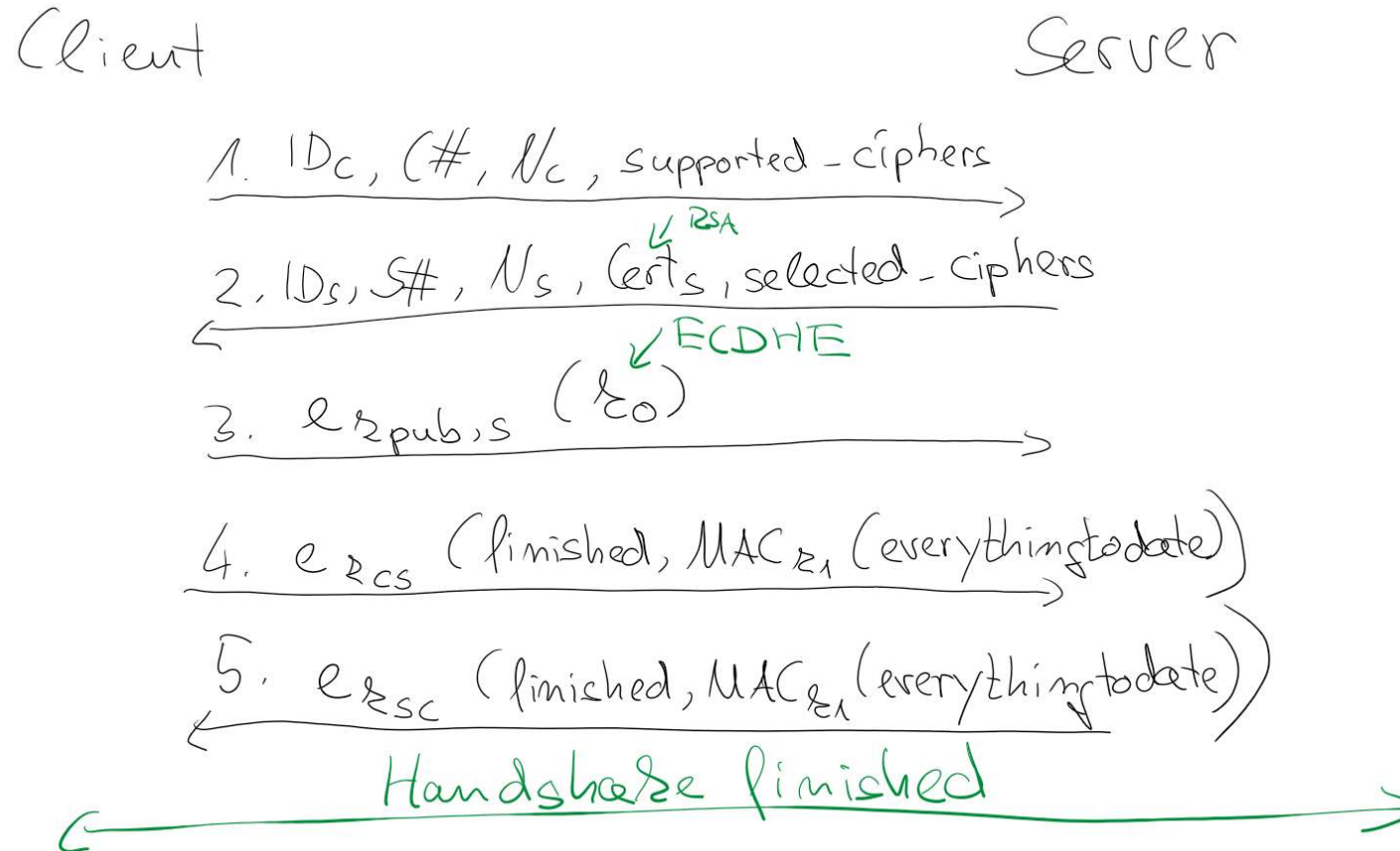
 **https://www.**

Historie gemäß heise online [[Link](#)]

- SSL 1.0: Aufgrund von Sicherheitsproblemen nie öffentlich freigegeben.
- SSL 2.0: Veröffentlicht 1995. Seit 2011 veraltet.
- SSL 3.0: Veröffentlicht 1996. Seit 2015 veraltet.
- TLS 1.0: 1999 als Upgrade auf SSL 3.0 veröffentlicht. Seit 2020 veraltet.
- TLS 1.1: Veröffentlicht 2006. Seit 2020 veraltet.
- TLS 1.2: Veröffentlicht 2008.
- TLS 1.3: Veröffentlicht 2018.

TLS 1.0 (Vereinfacht)

<https://datatracker.ietf.org/doc/html/rfc2246>



Erklärungen zu den Protokollnachrichten

Nachricht 1:

Client hello

Client ID: ID_C

Nonce N : N_C

Transaction Serial Number: $C\#$

supported-ciphers:

Nachricht 2

Server hello

Server ID: ID_S

Nonce N : N_S

$Cert_S = [k_{pub,s}, ID_S, sig_{CA}(k_{pub,s}, ID_S)]$

Selected ciphers

Nachricht 3

k_0 : pre-master-secret

RSA: k_0 vom Client gewählt

DHKE: k_0 wird auf Basis des
Parameteraustauschs berechnet

Nachricht 4:

$k_1 = PRF(k_0, N_C, N_S)$

↑

master-secret

↓

1. client - write - MAC - secret
2. server - write - MAC - secret
3. client - write - key (k_{cs})
4. server - write - key (k_{sc})

Denial-of-Service (DoS)

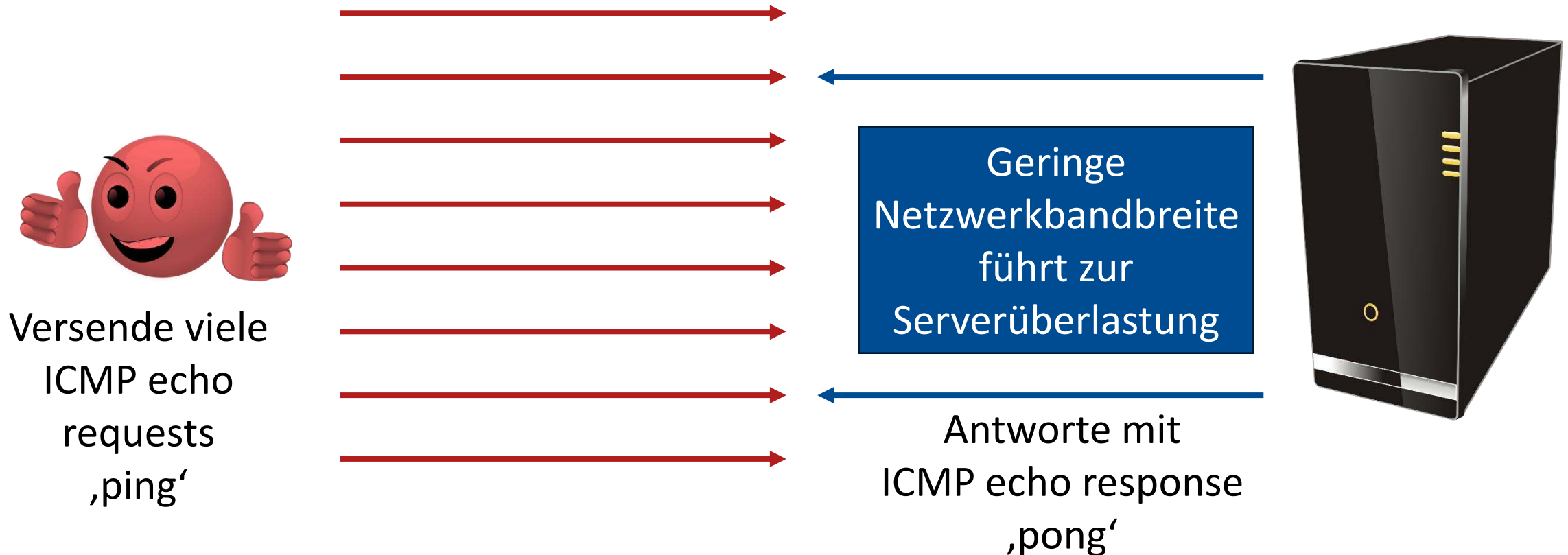
- DoS Angriffe zielen darauf ab das Sicherheitsprinzip von **Verfügbarkeit** (Availability) zu verletzen
- Bei DoS Angriffen geht es immer darum Ressourcen eines Systems zu überlasten so dass legitime Anfragen nicht mehr bearbeitet werden können
- Es gibt unterschiedliche Kategorien von Ressourcen, die angegriffen werden können:
 - Netzwerk Verbindungen (Network bandwidth)
 - System-Ressourcen
 - Applikation-Ressourcen

- Ausgewählte Beispiele:
 - 2008: YouTube für mehrere Stunden nicht erreichbar (*BGP Security*)
 - 2016: Mirai botnet erreicht, dass Twitter für 5 Stunden nicht erreichbar ist (*DNS Security*)
 - 2016: Krebsonsecurity.com (*Distributed DoS*)

- Das Border Gateway Protocol (BGP) wird von Routern genutzt, um Informationen über Routen auszutauschen
- Router können somit ‚Routing Tables‘ aktuell halten und die effizientesten Routen wählen
- Sicherheitsprobleme:
 - Wenn es einem Angreifer gelingt viele Router zu kapern, kann er falsche Routeninformationen verteilen
 - Beispiel: YouTube für mehrere Stunden 2008 nicht erreichbar aufgrund falscher BGP Informationen

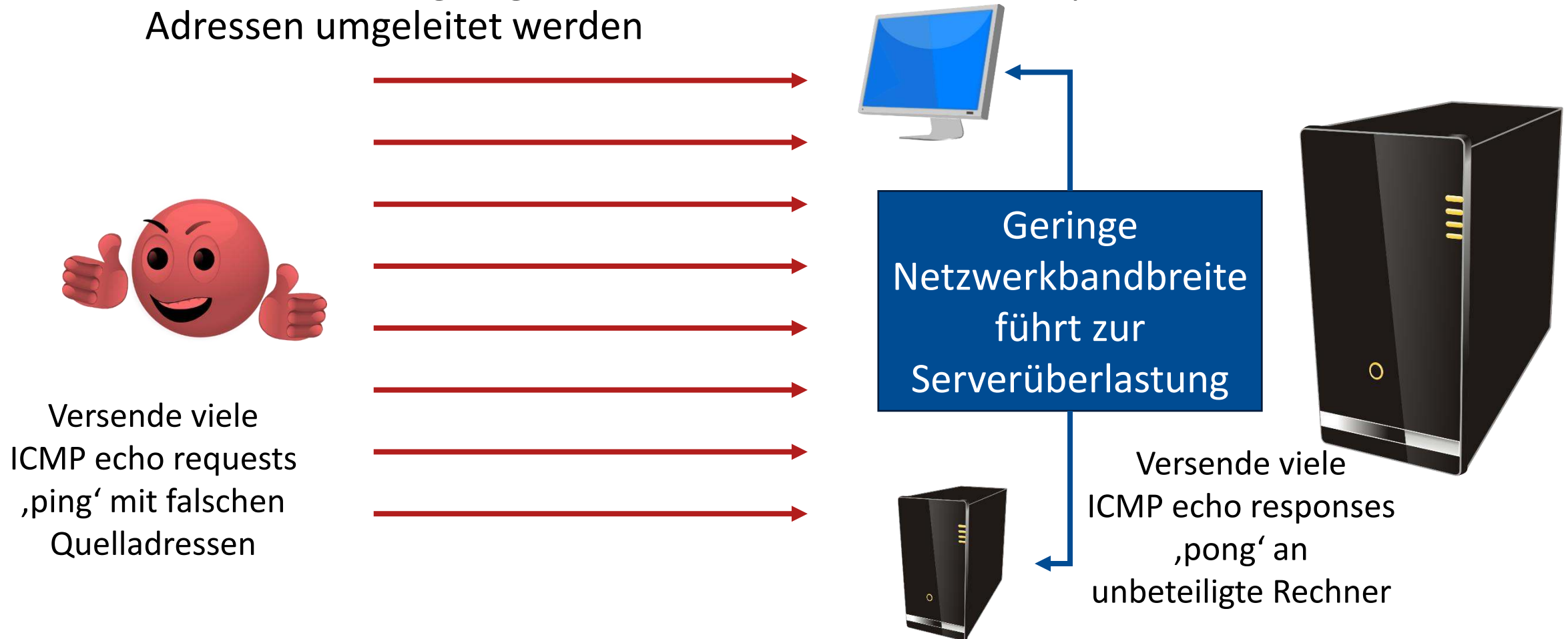
ICMP Flooding Angriff

Das Internet Control Message Protocol (ICMP) dient zum Austausch von Status- und Fehlermeldungen

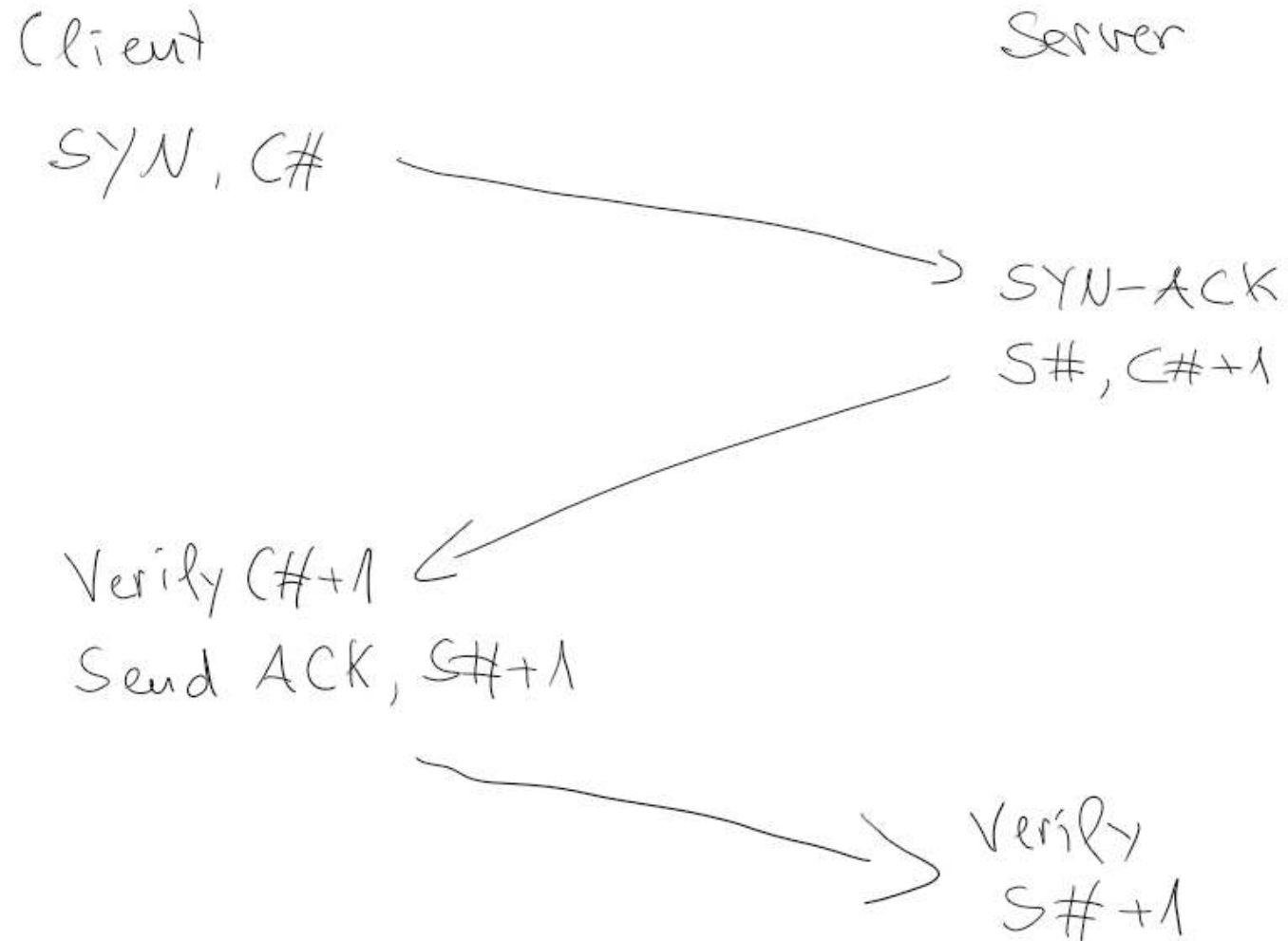


Source Address Spoofing

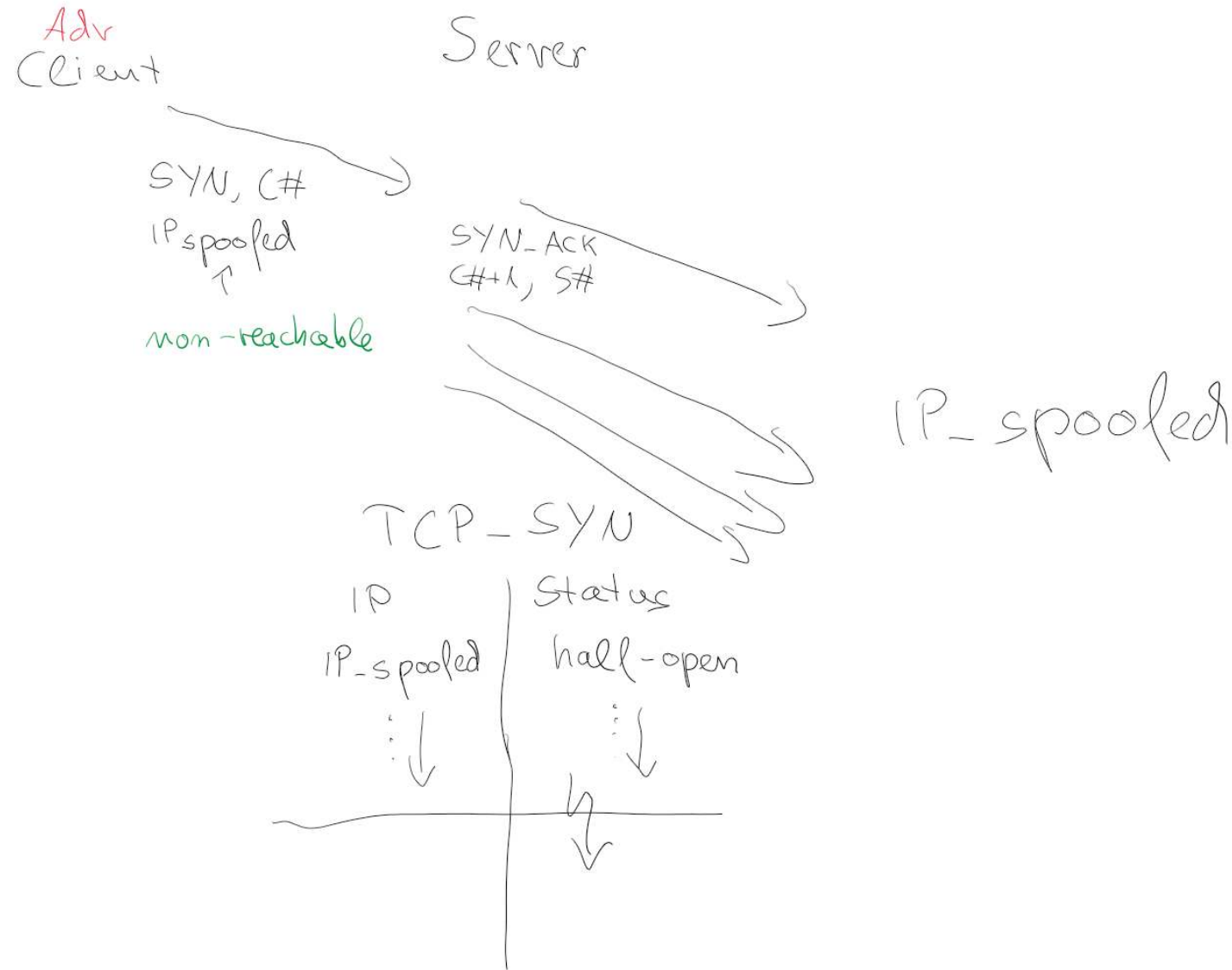
- Diese Technik erlaubt das Verschleiern der Quelladresse eines Angriffes
- Im ICMP Flooding Angriff könnten somit alle ICMP responses an willkürliche Adressen umgeleitet werden



SYN Spoofing: TCP Handshake

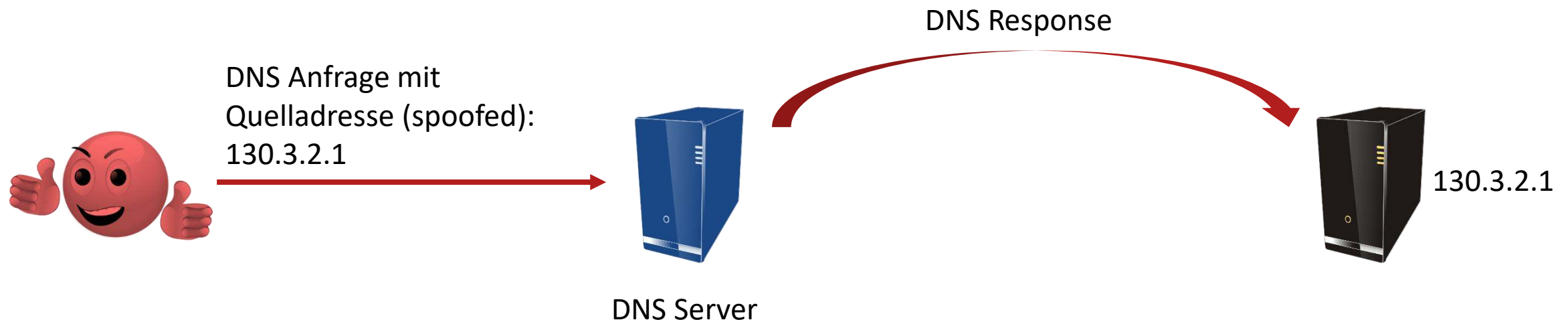


SYN Spoofing Angriff



Reflection Angriff

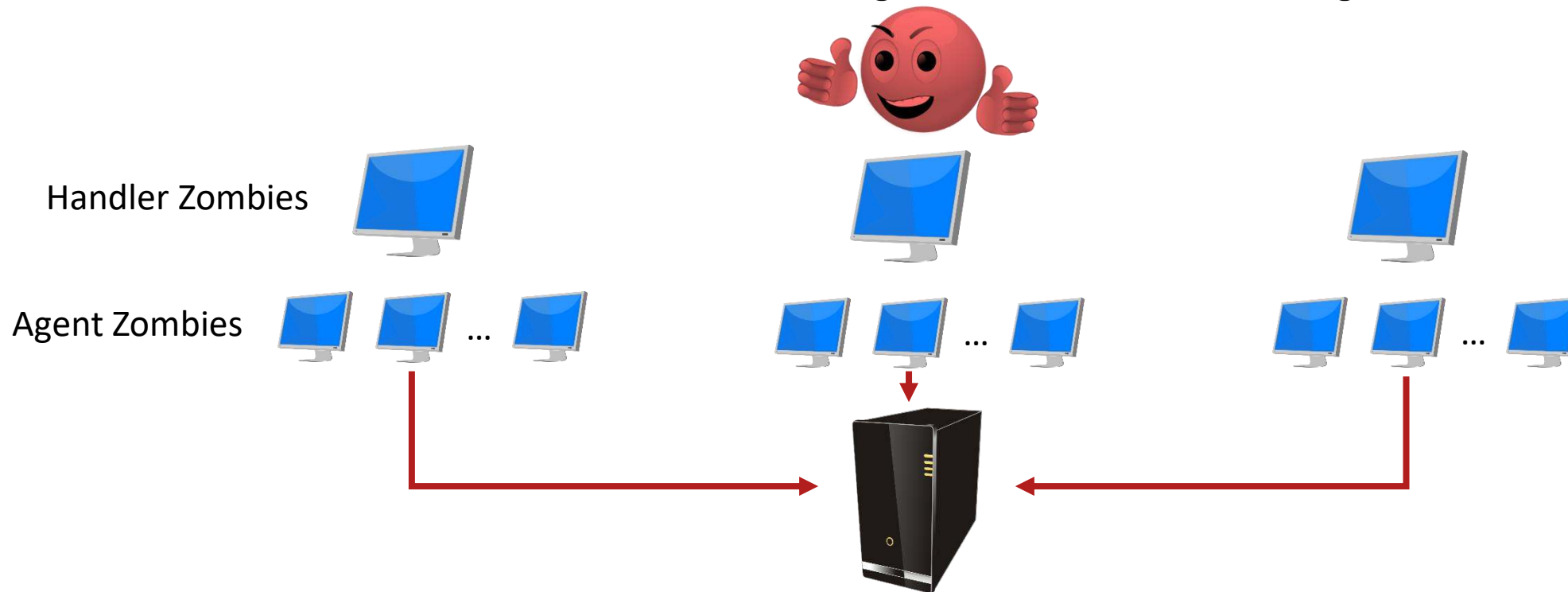
- Bei einem Reflection Angriff verschickt der Angreifer eine Anfrage mit gefälschter (spoofed) Quelladresse an verschiedene Server
- Die Antworten der Server gehen an die gefälschte Adresse und setzen das Opfersystem einem DoS aus
- Beispiel: DNS Reflection
 - Das Domain Name System (DNS) übersetzt Domain-Namen (uni-due.de) zu IP-Adressen
 - Bei einer DNS Reflection entsteht eine Endlosschleife zwischen DNS Server und Zielsystem



- Ein Amplification Angriff ist eine Variante des Reflection Angriffes bei der eine Request Nachricht zu mehreren Response Nachrichten führt
- Alternativ kann ein Amplification Angriff darauf aufsetzen, dass eine kleine Request-Nachricht zu einer deutlich größeren Response-Nachricht führt
- Beispiel: DNS Amplification
 - Beim DNS Protokoll kann eine 60-Byte UDP (User Datagram Protocol) Request-Nachricht zu einer 512-Byte UDP Response-Nachricht führen
 - Durch das Extended DNS (für IPv6) sind sogar Antworten bis 4000 Bytes möglich
 - Die DNS Amplification wird dadurch erreicht, dass der Angreifer in der Request-Nachricht viele ('ANY') DNS Informationen anfragt

Distributed DoS (DDoS)

- Die bis jetzt besprochenen DoS Angriffe können nur limitierten Traffic erzeugen, da sie von einem einzigen System aus durchgeführt werden
- Bei einem Distributed DoS (DDoS) Angriff werden stattdessen mehrere Angriffsrechner genutzt
 - Das Kollektiv mehrerer Rechner unter Angreifer Kontrolle wird häufig als Botnet bezeichnet



- Herausforderung:
 - Legitimes von Böartigen Überlasten unterscheiden
- Möglichkeiten für Abwehrmethoden
 - Bandbreite und Redundanz erhöhen
 - Angriffserkennung und Filtering (Firewalls)
 - Identifizierung des Angreifers

IoT Bots: The Case of Mirai

*Prof. Ahmad-Reza Sadeghi and Thien Nguyen
TU Darmstadt*

- IoT botnet **appearing** for the first time in **2016**
- **Infects** typical IoT devices: CCTV, DVRs, Home Routers, etc.
- **Actively scans** the network and **propagates** to any vulnerable devices it finds
- Has been used to perform the **largest DDoS attack** in history

Why is Mirai Possible?

Always online

Vulnerabilities

Weak
credentials

Poor network
management

Manufacturers **expose sensitive services**, e.g., Telnet access to their devices

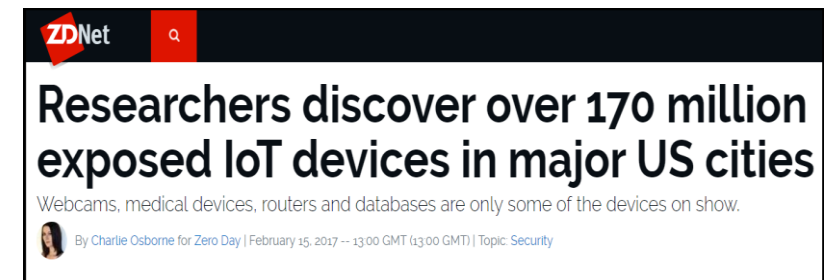
Poor default passwords
Password change at first use is not enforced

Users **do not have proper network configuration** leading to exposure of IoT devices on Internet

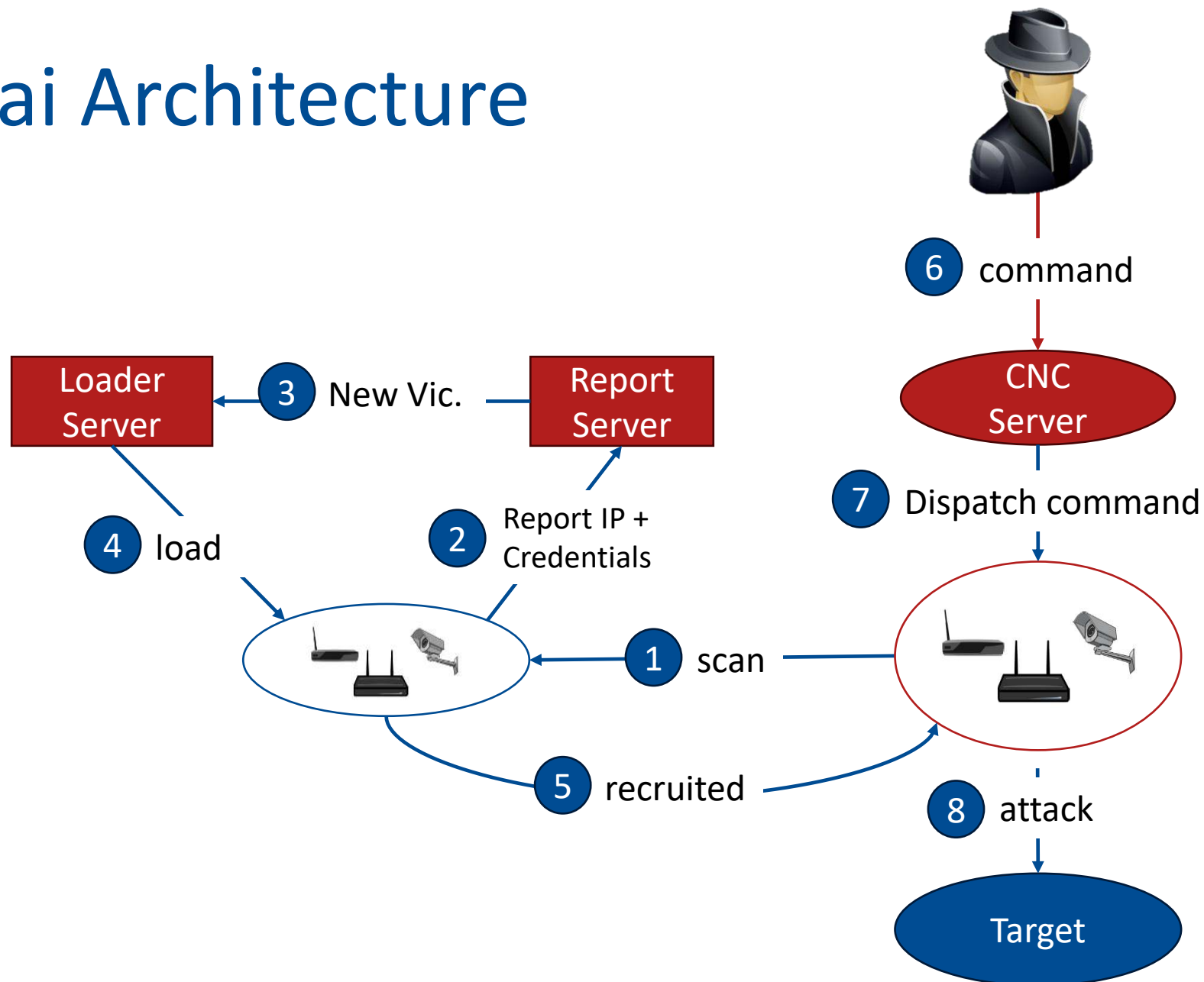
Examples of open ports

TCP/23
TCP/2323
TCP/23231
TCP/22
TCP/80
TCP/443

Username	Password	Device Type
root	system	IQinVision Cameras
root	00000000	Panasonic Printer
root	realtek	RealTek Routers
admin	1111111	Samsung IP Camera
admin	1111	Xerox printers, et. al
root	Zte521	ZTE Router



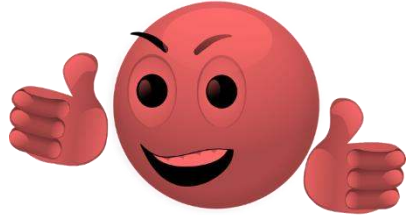
Mirai Architecture



Input Validation für Web Applikationen

Die folgenden Folien nutzen PHP Code. Wir werden in der Klausur nicht das Schreiben von PHP Code verlangen. Es kann aber durchaus vorkommen, dass PHP Code im Rahmen der gezeigten Beispiele analysiert werden muss. Auch das Auswendig Lernen von einzelnen PHP Funktionen ist nicht notwendig.

Input Validation Problem: Cross-Site Scripting



```
/index.php?name=test</h1>  
<script>alert(document.cookie)</script>
```

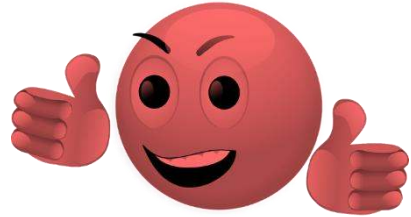


www.vulnerable.com



```
print('<h1>Welcome ' . $_GET['name'] . '</h1>');
```

Input Validation Problem: SQL Injection



```
/index.php?name=foo'+UNION+SELECT  
+password+FROM+users'
```



www.vulnerable.com



```
$name = $_GET['name'];  
$query = "SELECT birthday FROM users WHERE  
name = '$name'";
```

Wie kann man diese Angriffe verhindern?

Input Validation

```
graph TD; A[Input Validation] -.-> B[Sanitization and Transformation]; A -.-> C[Validation Based on Condition Checking];
```

Sanitization and
Transformation

Validation Based on
Condition Checking

- Explizites Typecasting
 - Explizites Casting einer Variable zu einem bestimmten Typ: integer, float, boolean, string, object

```
$number = (int)$number;  
$flag = (bool)$flag;
```

- Implizites Typecasting
 - Automatisches Casting auf Basis der Operationen die auf einer Variable ausgeführt werden

```
$number = $number + 1; // safe implicit type cast  
$string_var = $string_var++; // unsafe implicit type cast
```

Input Sanitization and Transformation : Encoding

- Angriffe nutzen häufig Sonderzeichen
- Idee: Lösche alle Sonderzeichen

```
$var = base64_encode($var);           // safe
$var = urlencode($var);               // safe

$var = zlib_encode($var, 15);          // unsafe
$var = urldecode($var);               // unsafe
```

Input Sanitization: Context-Sensitive Input Sanitization

- Lösche oder verändere bestimmte Zeichen

1. Konvertierung von Variablen

```
$var = htmlentities($var);  
echo '<a href=".php">'.$var.'</a>';
```

Ersetze '<' and '>'
mit **<** und **>**

2. Reguläre Ausdrücke (RegEx)

```
$var1 = preg_replace("/[^a-z0-9]/", "", $var1); // safe  
$var2 = preg_replace("/[^a-z.-_]/", "", $var2); // unsafe
```

Nur alphanumerische
Zeichen

Voller ASCII Bereich
zwischen Punkt und
Unterstrich erlaubt

Condition-Based Input Validation: Type Validation

- Sehr ähnlich zum expliziten Casting aber diesmal basierend auf tatsächlichen Überprüfungen

```
if(is_numeric($var)) { }
```

```
if(is_int($var) === true) { }
```

```
if($var = (int)$var) { }
```


Condition-Based Input Validation: Format Validation

- Prüfen ob Daten ein gewisses Datenformat einhalten

```
if(checkdate($var)) { }
```

```
if($var = strtotime($var)) { }
```

Condition-Based Input Validation: Whitelisting

- Blacklisting ist fehleranfällig da man eventuell verpasst alle gefährlichen Inputs zu erfassen
- Whitelisting wird als sicherer angesehen, da die Eingabe auf einen bestimmten Bereich eingegrenzt wird

```
$whitelist = array('a' => true, 'b' => true, 'c' => true);
```

```
if(isset($whitelist[$var])) { }
```

```
if($whitelist[$var]) { }
```

```
if(array_key_exists($var, $whitelist)) { }
```

- CSP is a W3C standard that offers mitigation against malicious input in web applications
 - Policy language to restrict content
 - Whitelist for legitimate content inclusion
 - Inline scripts and styles are blocked by default
 - Dangerous function such as eval prohibited by default
- However, CSP does not substitute input validation

Policy example for <https://my-secure-zone.com>

```
script src https://\*facebook.net;  
default-src 'self';
```

Allowed Behavior

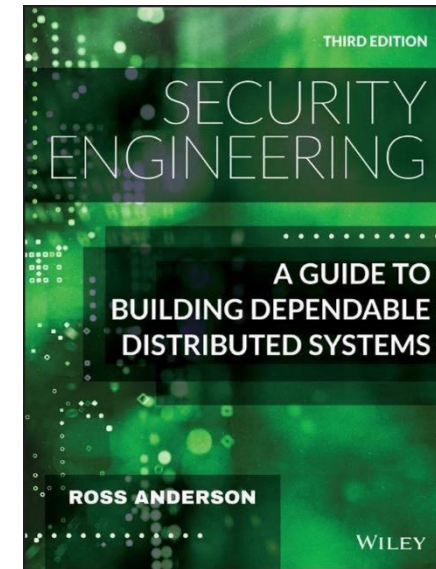
- Include scripts from <https://connect.facebook.net>
- Include images, styles from <https://my-secure-zone.com>

Blocked Behavior

- Include scripts from <https://malicious.com>
- Include images from <http://my-secure-zone.com>
- Run inline scripts

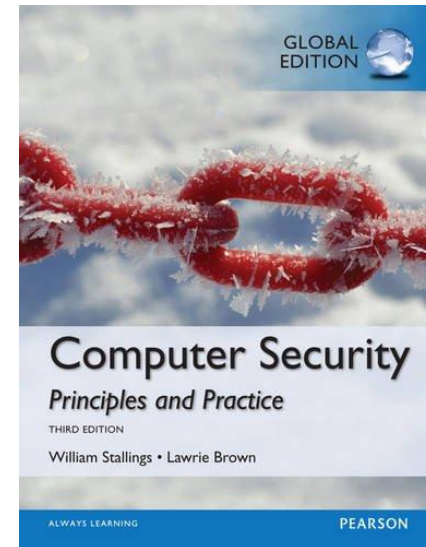
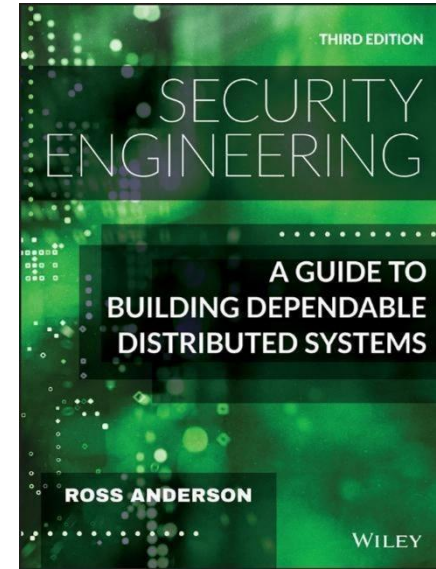
- Die Vorlesungsfolien für dieses Thema sind auf Basis des Buchs „Security Engineering“ von R. Anderson entstanden
- Für TLS 1.0 wurden einige Erklärungen aus dem RFC entnommen:

<https://datatracker.ietf.org/doc/html/rfc2246>



Referenzen für DoS

- Die Vorlesungsfolien für dieses Thema sind auf Basis des Buchs „Security Engineering“ von R. Anderson und „Computer Security“ von W. Stallings und L. Brown entstanden



- Input Validation
 - Johannes Dahse. *Static Detection of Complex Vulnerabilities in Modern PHP Applications*. PhD Dissertation at RUB. 2015
- CSP Analyse
 - Calzavara, Stefano and Rabitti, Alvis and Bugliesi, Michele. *Content Security Problems?: Evaluating the Effectiveness of Content Security Policy in the Wild*. ACM CCS 2016

Vorlesung Cybersicherheit, Sommersemester 2024

Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi
Fakultät für Informatik
Arbeitsgruppe Systemsicherheit
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi